

Functional Programming

Course: Rust Basics

Author: Aleksandr Kugushev

Functional Programming

- Functions are treated as first-class citizens
- ~~Pure functions~~
- Recursion (<https://docs.rs/tailcall/latest/tailcall/>)
- Lazy evaluation
- Discriminated Unions
- Type inference (based on the standard Hindley-Milner)

fn pointer

```
fn fun_fun(i: i32) -> String {  
    i.to_string()  
}  
  
let fun: fn(i32) -> String = fun_fun;  
first_class(fun);  
  
fn first_class(func: fn(i32) -> String) {  
    let output: String = func(42);  
    println!("{}", output)  
}
```

fn pointer

```
let func: fn(i32) -> String =  
    |i: i32| -> String {  
        i.to_string()  
    };  
first_class(func);
```

Lambda syntax

```
let fun: fn() = || {};
```

```
let fun: fn(i: i32, j: i32) = |i: i32, j: i32| {};
```

```
let fun: fn() -> i32 = || -> i32 { 42 };
```

```
let fun: fn() -> i32 = || 42 ;
```

Closure

```
let a = String::from("Hi");
```

```
let func = |i: i32| -> String { a };
```

```
first_class(func);
```

^^^^ expected fn pointer, found closure

Closure is a trait impl

```
let a = String::from("Hi");  
let func = |i: i32| -> String { a };  
first_class(func);
```

```
fn first_class(  
    func: impl FnOnce() -> String) {  
    let output: String = func();  
    println!("{}", output)  
}
```

// not a real code

```
struct MyClosure{  
    func: fn(args: (), payload: String) -> String,  
    payload: String  
}  
  
impl FnOnce<()> for MyClosure{  
    type Output = String;  
    fn call_once(self, args: ()) -> Self::Output {  
        self.func(args, self.payload)  
    }  
}
```

Closure is a generic

```
pub trait FnOnce<Args: Tuple>
{
    type Output;

    fn call_once(self, args: Args)
        -> Self::Output;
}
```

```
fn closure(func: impl FnOnce(i32, i32) -> u8) {
    let byte: u8 = func(1, 2);
}
```

```
fn closure(func: impl FnOnce<(i32, i32),
Output=u8>) {
    let byte: u8 = func(1, 2);
}
```

the precise format of `Fn`-family traits' type parameters is subject to change

FnOnce trait

```
pub trait FnOnce<Args: Tuple> {  
    type Output;  
  
    fn call_once(self, args: Args) -> Self::Output;  
}  
  
fn first_class(func: impl FnOnce() -> String) {  
    let output: String = func();  
    func(); <- value used here after move  
    println!("{}", output)  
}
```

Fn trait

```
pub trait Fn<Args: Tuple>: FnMut<Args> {  
    fn call(&self args: Args) -> Self::Output;  
}
```

```
fn first_class_by_ref(func: impl Fn() -> String) {  
    let output: String = func();  
    func();  
    println!("{}", output)  
}
```

Fn trait call

```
let a = String::from("Hi");  
let func = || -> String {a};  
first_class_by_ref(func);
```

error[E0525]: expected a closure
that implements the `Fn` trait, but
this closure only implements
`FnOnce`

```
fn first_class_by_ref(  
    func: impl Fn() -> String) {  
  
    let output: String = func();  
    func();  
    println!("{}", output)  
}
```

Fn trait call

```
let a = String::from("Hi");  
let func = || -> String {  
    a.to_string()  
};  
first_class_by_ref(func);
```

```
fn to_string(&self) -> String
```

```
fn first_class_by_ref(func: impl Fn()  
-> String) {  
    let output: String = func();  
    func();  
    println!("{}", output)  
}
```

Fn trait call

```
let a = String::from("Hi");  
let func = || -> String {  
    a.to_string()  
};  
drop(a);
```

error[E0505]: cannot move out
of `a` because it is borrowed

Fn trait call and mutation

```
let a = String::from("Hi");  
let func = || -> String {  
    a.push_str(" World");  
    a.to_string()  
};  
first_class_by_ref(func);
```

error[E0525]: expected a closure
that implements the `Fn` trait, but
this closure only implements
`FnMut`

```
fn first_class_by_ref(func: impl  
Fn() -> String) {  
    let output: String = func();  
    func();  
    println!("{}", output)  
}
```

FnMut trait

```
pub trait FnMut<Args: Tuple>: FnOnce<Args> {  
    fn call_mut(&mut self, args: Args) -> Self::Output;  
}
```

```
fn first_class_by_mutref(mut func: impl FnMut() -> String) {  
    println!("{}", func()); // > Hi World  
    println!("{}", func()); // > Hi World World  
}
```

FnMut trait

```
let mut a = String::from("Hi");  
let func = || -> String {  
    a.push_str(" World");  
    a.to_string()  
};  
first_class_by_mutref(func);  
println!("{}", a); // > Hi World  
World
```

```
fn first_class_by_mutref(mut func: impl  
FnMut() -> String) {  
    println!("{}", func()); // > Hi World  
    println!("{}", func()); // > Hi World W  
}
```


Traits relation

`Fn<Args>: FnMut<Args>: FnOnce<Args>`

`FnOnce`

`fn call_once(self, args: Args) -> Self::Output;`

`FnMut`

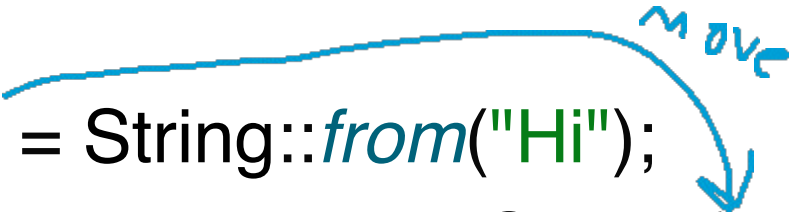
- `fn call_once(self, args: Args) -> Self::Output;`
- `fn call_mut(&mut self, args: Args) -> Self::Output;`

`Fn`

- `fn call_once(self, args: Args) -> Self::Output;`
- `fn call_mut(&mut self, args: Args) -> Self::Output;`
- `fn call(&self, args: Args) -> Self::Output;`

Why FnOnce declared call_once only?

```
let a = String::from("Hi");  
let func = |i: i32| -> String { a };  
first_class(func);
```



```
fn call_once(self, args: Args) ->  
Self::Output;
```



```
fn first_class(  
func: impl FnOnce() -> String) {  
    let output: String = func();  
    println!("{}", output)  
}
```



`move || {...}`

```
let func = return_print_42();  
func();
```

```
fn return_print_42() -> impl Fn(){  
    let int = 42;  
    || println!("{int}")  
}
```

closure may outlive the current function, but it borrows `int`, which is owned by the current function

move || {...}

```
fn return_print_42() -> impl Fn(){  
    let int = 42;  
    move || println!("{int}")  
}
```

Practice: Functions

```
execute1(/* no closure required, just  
execute to_string() */);
```

```
let mut my_string =  
String::from("something");
```

```
execute2(/* you should return  
my_string.len() in lambda */);
```

```
execute3(/* execute  
my_string.push_str(&append) in lambda  
*/);
```

```
execute4(/* use move semantic */);
```

```
fn execute1(func: ???) {  
    let result = func(42);  
    assert_eq!("42", result);  
}
```

```
fn execute2(func: ???) {  
    let string_len = func();  
    assert_eq!(9, string_len)  
}
```

```
fn execute3(mut func: ???) {  
    func(String::from(" good"));  
}
```

```
fn execute4(func: ???) {  
    let result: String = func();  
    assert_eq!("something good", result);  
}
```

Destructuring

```
let tuple = (1, 2);  
let (x, y) = tuple;
```

```
let [first, second] = [1, 2];
```

```
let s = MyStruct { name: String::from("Name"), integer: 42 };  
let MyStruct { name, integer } = s;
```

```
fn destruct((x, y): (i32, i32), MyStruct { name, integer }: MyStruct) {}
```

Destructuring and wildcard

```
let tuple = (1, 2);  
let (x, _) = tuple;
```

```
let [_, second] = [1, 2];
```

```
let s = MyStruct { name: String::from("Name"), integer: 42 };  
let MyStruct { name, .. } = s;
```

```
fn destruct((x, _): (i32, i32), MyStruct { name, .. }: MyStruct) {}
```

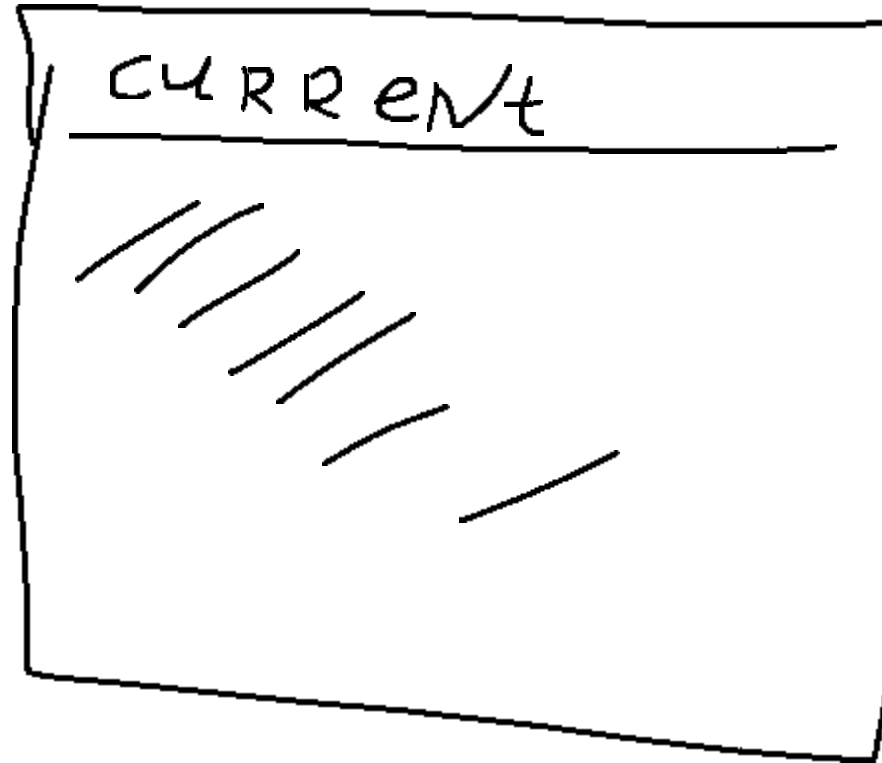
Discriminated union

a union data structure that holds various objects, with one of the objects identified directly by a discriminant. (ibm)

```
enum Employee{  
    Merchandiser,  
    Cashier,  
    Laborer,  
    Supervisor,  
    Director,  
}
```


Enum a union of structs

```
enum Employee{  
    Merchandiser,  
    Cashier,  
    Laborer,  
    Supervisor,  
    Director,  
}
```



Enum impl

```
impl Employee {  
    fn praise(&self) {  
        println!("Give me money, no praise!");  
    }  
}
```

```
impl Employee::Director {  
    fn bribe() {}  
}
```

error[E0573]: expected type, found variant `Employee::Director`

Enum and Pattern Matching

```
fn fire(employee: Employee) {  
  match employee {  
    Employee::Merchandiser => { println!("Ok :`(") }  
    Employee::Cashier => { println!("Ok :`(") }  
    Employee::Laborer => { println!("Ok :`(") }  
    Employee::Supervisor => { println!("Ok :`(") }  
    Employee::Director => { println!("No, YOU'RE FIRED!") }  
  }  
}
```

Enum a union of structs

```
enum Employee {  
    Merchandiser { name: String, age:  
u8 },  
    Cashier(String),  
    Laborer(u32, u32),  
    Supervisor(Degree),  
    Director,  
}  
enum Degree{  
    School,  
    University  
}
```

```
fn pay_salary(employee: Employee) {  
    match employee {  
        Employee::Merchandiser  
{ name, age } => {},  
        Employee::Cashier(name) => {},  
        Employee::Laborer(height,  
width) => {},  
        Employee::Supervisor(degree)  
=> {},  
        Director => {}  
    }  
}
```

Enum instantiate

```
pay_salary(Employee::Merchandiser {  
    name: String::from("Ivan"),  
    age: 42,  
});  
pay_salary(Employee::Cashier(String::from("Maria")));  
pay_salary(Employee::Director);
```

Pattern Matching

```
fn pay_salary(employee: Employee) -> u32 {  
    match employee {  
        Employee::Merchandise { name, age } => {  
            if age > 18 { 10 } else { 1 }  
        }  
        Employee::Cashier(_) => { 5 }  
        Employee::Laborer(height, _) => { height }  
        Employee::Supervisor(degree) => {  
            match degree {  
                Degree::School => { 20 }  
                Degree::University => { 40 }  
            }  
        }  
        Employee::Director => { 1000 }  
    }  
}
```

Pattern Matching is exhaustive

```
fn pay_salary(employee: Employee) -> u32 {  
  match employee {  
    Employee::Merchandise { name, age } => {  
      if age > 18 { 10 } else { 1 }  
    }  
    Employee::Cashier(_) => { 5 }  
    Employee::Laborer(height, _) => { height }  
    Employee::Supervisor(degree) => {  
      match degree {  
        Degree::School => { 20 }  
        Degree::University => { 40 }  
      }  
    }  
    // Employee::Director => { 1000 }  
  }  
}
```

error[E0004]: non-exhaustive patterns: `fp::Employee::Director` not covered

Match value

```
fn pay_salary(employee: Employee) -> u32 {  
  match employee {  
    Employee::Merchandise { name, age } => {  
      if age > 18 { 10 } else { 1 }  
    }  
    Employee::Cashier(_) => { 5 }  
    Employee::Laborer(180, _) => { 200 }  
    Employee::Supervisor(degree) => {  
      match degree {  
        Degree::School => { 20 }  
        Degree::University => { 40 }  
      }  
    }  
    Employee::Director => { 1000 }  
  }  
}
```

error[E0004]: non-exhaustive patterns: `fp::Employee::Laborer(0\_u32..=179\_u32, \_)` and
`fp::Employee::Laborer(181\_u32..=u32::MAX, \_)` not covered

Match value

```
fn pay_salary(employee: Employee) -> u32 {  
  match employee {  
    Employee::Merchandiser { name, age } => {  
      if age > 18 { 10 } else { 1 }  
    }  
    Employee::Cashier(_) => { 5 }  
    Employee::Laborer(180, _) => { 200 }  
    Employee::Laborer(..=180, _) => { 100 }  
    Employee::Supervisor(degree) => {  
      match degree {  
        Degree::School => { 20 }  
        Degree::University => { 40 }  
      }  
    }  
    Employee::Director => { 1000 }  
  }  
}
```

Range

start..end

start..

..end

..

start..=end

..=end

[std::ops::Range](#)

[std::ops::RangeFrom](#)

[std::ops::RangeTo](#)

[std::ops::RangeFull](#)

[std::ops::RangeInclusive](#)

[std::ops::RangeToInclusive](#)

$\text{start} \leq x < \text{end}$

$\text{start} \leq x$

$x < \text{end}$

-

$\text{start} \leq x \leq \text{end}$

$x \leq \text{end}$

Match value

```
fn pay_salary(employee: Employee) -> u32 {  
  match employee {  
    Employee::Merchandiser { name, age } => {  
      if age > 18 { 10 } else { 1 }  
    }  
    Employee::Cashier(_) => { 5 }  
    Employee::Laborer(height, _) => { height }  
    Employee::Supervisor(degree) => {  
      match degree {  
        Degree::School => { 20 }  
        Degree::University => { 40 }  
      }  
    }  
    Employee::Director => { 1000 }  
  }  
}
```

Match value

```
fn pay_salary(employee: Employee) -> u32 {  
    match employee {  
        Employee::Merchandiser { name, age } => {  
            if age > 18 { 10 } else { 1 }  
        }  
        Employee::Cashier(_) => { 5 }  
        Employee::Laborer(180.., _) => { 200 }  
        Employee::Laborer(..=180, _) => { 200 }  
        Employee::Supervisor(Degree::School) => { 20 }  
        Employee::Supervisor(Degree::University) => { 40 }  
        Employee::Director => { 1000 }  
    }  
}
```

or

```
fn pay_salary(employee: Employee) -> u32 {  
  match employee {  
    Employee::Merchandiser { name, age } => {  
      if age > 18 { 10 } else { 1 }  
    }  
    Employee::Cashier(_) => { 5 }  
    Employee::Laborer(180.., _) => { 200 }  
    Employee::Laborer(..=180, _) => { 200 }  
    Employee::Supervisor(Degree::School | Degree::University) => { 30 }  
    Employee::Director => { 1000 }  
  }  
}
```

Pattern Matching for Structs

```
struct Customer {  
    name: String,  
    age: u8,  
}  
  
match customer  
{  
    Customer { name, age: ..=17 } => {  
        println!("Don't buy beer, dear {}", name)  
    }  
    Customer { age: 17..., .. } => {  
        println!("Enjoy your lager")  
    }  
    Customer { .. } => {}  
}
```

Pattern Matching for Tuples

```
let product = (String::from("Milk"), 4, true);  
match product {  
    (name, 4, _) => {}  
    (name, ..) => {}  
}
```

Match Guard

```
let num = Some(4);
```

```
match num {  
  Some(x) if x % 2 == 0 => println!("The number {} is even", x),  
  Some(x) => println!("The number {} is odd", x),  
  None => (),  
}
```


Match Guard and exhaustive check

```
let num = Some(4);  
match num {  
    Some(x) if x > 2 => println!("The number {} is more than 2", x),  
    Some(x) if x <= 2 => println!("The number {} is less or equal than  
2", x),  
    None => (),  
}
```

error[E0004]: non-exhaustive patterns: `Some(\_)` not covered

@ binding

```
let num = Some(4);  
match num {  
    Some(x @ 2..) => println!("The number {} is more than 2", x),  
    Some(x @ ..=2) => println!("The number {} is less or equal than  
2", x),  
    None => (),  
}
```

Wildcard

```
fn pay_salary(employee: Employee) -> u32 {  
    match employee {  
        Employee::Director => { 1000 },  
        _ => 10  
    }  
}
```

if let

```
fn pay_salary(employee: Employee) -> u32 {  
    if let Employee::Director = employee {  
        return 1000  
    }  
    10  
}  
  
fn is_tall_laborer(employee: Employee) -> bool {  
    if let Employee::Laborer(..=180, _) = employee { true } else  
{ false }  
}
```

while let

```
let mut stack = Vec::new();
```

```
stack.push(1);  
stack.push(2);  
stack.push(3);
```

```
while let Some(top) = stack.pop() {  
    println!("{}", top);  
}
```

Option

```
pub enum Option<T> {  
    None,  
    Some(T),  
}
```

Use Option

```
struct Node {  
    value: i32,  
    next: Option<Box<Node>>,  
}  
  
fn visit(node: Box<Node>) {  
    println!("{}", node.value);  
    match node.next {  
        Some(next) => { visit(next) }  
        None => { println!("Done"); }  
    }  
}
```

Use Option by ref

```
fn visit(node: Box<Node>) {  
    println!("{}", node.value);  
    match node.next {  
        Some(next) =>  
        { visit(next) }  
        None => { println!  
("Done"); }  
    }  
}
```

```
fn visit_ref(node: &Box<Node>) {  
    println!("{}", node.value);  
    match &node.next {  
        Some(next) => { visit_ref(next) }  
        None => { println!("Done"); }  
    }  
}
```


Practice: Discriminated Union

- CardId(string)
- CryptoNetwork
 - BTC
 - ETH
 - ScamNetwork
 - Other(string)
- WalletAddress(string)
- CheckInfo(owner: string, bank: string, dueTime: u64)
- PaymentMethod
 - Cash
 - BankCard(CardId)
 - Crypto(CryptoNetwork, WalletAddress)
 - Check(CheckInfo)

Homework

- Rustlings:
 - functions
 - enums
- Coursware:
 - Support in text search
 - Add --in-file key for .txt and .rs files
 - Refactor:
 - Create enum Occurrence
 - File
 - Directory
 - TextFile
 - At first, scan directory and fill Occurrences
 - Second, go through Occurrences and apply filtering/sorting